

1. **What REST is:** What REST is · Stateless requests · REST, GraphQL and other API styles
2. **Designing URLs:** Naming resources: nouns, plurals, ids and nesting · Filters, sorting and pagination: the query string · Versioning: changing an API without breaking its clients
3. **Methods and CRUD:** CRUD on HTTP: the routes of a resource · Safe and idempotent: why retries matter · PUT replaces, PATCH changes
4. **Status codes and errors:** Success codes: 200, 201 and 204 · Client errors: choosing the right 4xx · 401, 403 or 404: who are you, and may you? · One error shape, and when to use 500
5. **The API contract:** JSON in, JSON out: Content-Type and Accept · The API contract vs the database row (DTOs) · Documenting the API: OpenAPI
6. **Building and testing:** CRUD with an in-memory array · Validating input: 400 with field errors · Testing by hand: curl, Postman, Thunder Client · Automated API tests with supertest

What REST is

What REST is

REST (Representational State Transfer) is a style of designing HTTP APIs: the server exposes resources at URLs, and clients read and change them by exchanging representations (usually JSON) through the same few HTTP methods, with every request standing on its own.

REST vocabulary

Term	Meaning	Example
Resource	A thing the API lets you name	a task, a user, a list of tasks
Collection	The resource that contains many items	/api/tasks
Item (member)	One resource inside a collection	/api/tasks/7
Representation	The resource written in a format, sent over HTTP	{"id":7,"title":"Study","done":false}
Endpoint	One method + path pair your API supports	DELETE /api/tasks/:id
Uniform interface	The same methods and codes mean the same everywhere	DELETE always removes, 404 always means "not there"

Example. The same features designed twice. Action style: POST /getTask with {"id":7}, POST /markTaskDone?id=7, POST /removeTask?id=7. REST style: GET /api/tasks/7, PATCH /api/tasks/7 with {"done":true}, DELETE /api/tasks/7. In the REST version a newcomer who has seen GET /api/users/3 correctly guesses DELETE /api/users/3; in the action version they must read the documentation for every single call.

Common mistake. Believing an API is RESTful because it answers JSON. POST /api/doEverything?action=delete&id=7 answers JSON and breaks every REST rule: the URL names an action, the method is meaningless and the status code tells you nothing. REST is about resources and a uniform interface, not about the format (a REST API could answer XML).

Stateless requests

In a **stateless API** each request carries everything the server needs to handle it (who is calling, which resource, which options); the server keeps no memory of earlier requests from that client.

```
// X Stateful: the server keeps "who is logged in" in its own memory
let currentUser = null;
app.post('/api/login', (req, res) => {
  currentUser = req.body.email;           // shared by EVERY client of this process
  res.sendStatus(204);
});
app.get('/api/my-books', (req, res) => {
  res.json(books.filter((b) => b.owner === currentUser));
});

// ✓ Stateless: every request says who it comes from
app.get('/api/my-books', requireToken, (req, res) => {
```

```
res.json(books.filter((b) => b.owner === req.user.email)); // req.user set from the token
});
```

Example. With the stateful code above, Ana logs in, then Leo logs in: `currentUser` is now Leo, and Ana's next GET `/api/my-books` returns **Leo's** books. Run two copies of the server and it gets worse: Ana logged in on copy A, her next request lands on copy B, where `currentUser` is `null`. In the stateless version each request carries `Authorization: Bearer <token>`, so it does not matter which copy answers or who logged in last.

Common mistake. Thinking "stateless" forbids a database, or forbids remembering a logged-in user. The data lives in the database (resource state), and being logged in is remembered by the **client**, which sends its token with every request. Only per-client memory **inside the server process** is ruled out.

REST, GraphQL and other API styles

GraphQL is an alternative to REST in which the client sends a **typed query** to one endpoint (`/graphql`) and gets exactly the fields it asked for; REST has many URLs, each with a fixed response shape. It is worth recognising; REST is what the examples here build.

Other API styles (context only)

Style	How it talks	Good for
REST	HTTP methods on resource URLs, usually JSON	The default: simple, cacheable, every tool understands it
GraphQL	Typed queries to one HTTP endpoint	Screens that need data from many resources at once
gRPC	Compact binary messages over HTTP/2, typed contracts	Fast calls between a company's own back-end services
WebSockets	One connection that stays open, both sides can send	Chat, live notifications, multiplayer games
Webhooks	The server calls your URL when something happens	"Tell me when the payment is completed"

GraphQL

```
# One request: POST /graphql
query {
  user(id: 2) {
    name
    tasks(done: false) { title }
  }
}

# The answer (status 200 OK, even when part of it failed)
{ "data": { "user": { "name": "Leo Martín",
  "tasks": [ { "title": "Return the right status codes" } ] } } }
```

Example. A dashboard shows a user's name and the titles of their unfinished tasks. REST: GET `/api/users/2` and GET `/api/users/2/tasks?done=false`, two requests, each returning all fields. GraphQL: the single query above, returning only name and title. For a simple client (one list, one resource) the REST version is simpler and the browser can cache it.

Common mistake. Thinking GraphQL is a database or "REST but newer". GraphQL is a query language for an **API**: the server still runs functions (resolvers) that read the database. Most projects start with REST and add GraphQL only where flexible queries pay off.

Designing URLs

Naming resources: nouns, plurals, ids and nesting

A RESTful URL names a **thing**, never an action: a plural noun for the collection (`/api/tasks`), the collection plus an id for one item (`/api/tasks/7`), and a nested path when a resource lives inside another (`/api/users/2/tasks`).

From action URLs to resource URLs

Instead of	Write	Why
GET <code>/api/getAllTasks</code>	GET <code>/api/tasks</code>	The method already says "get"
POST <code>/api/createTask</code>	POST <code>/api/tasks</code>	Create = POST on the collection

Instead of	Write	Why
GET /api/task/7	GET /api/tasks/7	One plural noun for collection and items
GET /api/tasks?id=7	GET /api/tasks/7	Identity belongs in the path
POST /api/tasks/7/delete	DELETE /api/tasks/7	The method is the action
PUT /api/tasks/7/markDone	PATCH /api/tasks/7 + {"done":true}	Changing a field is an update
GET /api/tasksOfUser?user=2	GET /api/users/2/tasks OR GET /api/tasks?userId=2	Nesting or a filter, not a new name

Example. A book club API: GET /api/books (all books), GET /api/books/12 (one book), POST /api/books (add one), GET /api/books/12/reviews (the reviews of book 12), POST /api/books/12/reviews (review book 12), DELETE /api/reviews/88 (one review, by its own id). Six endpoints, two nouns, and every method means what it always means.

Common mistake. Adding a verb "because it is clearer", e.g. GET /api/books/search. It reads well, but now /api/books/search collides with /api/books/:id (Express may take search as an id) and clients must memorise one more name. A search is a filter on the collection: GET /api/books?search=dune.

Filters, sorting and pagination: the query string

The path says **which resource**; the query string says **which part of it, in what order and how much**: **filters** (?done=false), **search** (?search=css), **sorting** (?sort=createdAt&order=desc) and **pagination** (?page=2&limit=10).

```
// req.query in Express holds the same thing: every value is a STRING
const params = new URLSearchParams('done=false&page=2&limit=500&tag=a&tag=b');

console.log(params.get('done'), typeof params.get('done'));
console.log(Boolean(params.get('done'))); // the string "false" is truthy!
console.log(params.get('done') === 'true'); // the right test

const page = Math.max(1, Number(params.get('page')) || 1);
const limit = Math.min(100, Number(params.get('limit')) || 20); // clamped
console.log({ page, limit, offset: (page - 1) * limit });

console.log(params.get('tag'), params.getAll('tag')); // repeated keys
```

Example. With five tasks of which 2, 3 and 4 are unfinished, GET /api/tasks?done=false&sort=createdAt&order=desc&limit=2 returns 200 with tasks 4 and 3 (the two newest unfinished ones) and X-Total-Count: 3, so the client knows a second page exists. GET /api/tasks?done=false&page=5&limit=2 returns 200 with []: the page is past the end, not "not found".

Common mistake. Writing if (req.query.done) list = list.filter((t) => t.done). Query values are strings, and "false" is a non-empty string, so ?done=false returns the **finished** tasks. Compare with the string: req.query.done === "true", and reject anything that is neither "true" nor "false".

Versioning: changing an API without breaking its clients

A **version** in the URL (/api/v1/tasks) or in a header lets you publish an incompatible change as a new version while existing clients keep using the old one.

Example. Version 1 returns { "id": 7, "name": "Study" } and you want the field to be called title. A breaking rename would crash every client that reads task.name. Instead: in v1, send **both** name and title (adding is non-breaking); publish /api/v2/tasks with only title; mark v1 deprecated; switch it off once the clients have moved.

Common mistake. Renaming a field "to tidy up" inside the same version, or creating a new version for every small addition (/v1 ... /v14). Additions need no version; only breaking changes do.

Methods and CRUD

CRUD on HTTP: the routes of a resource

CRUD (Create, Read, Update, Delete) names the four things almost every API does to its data; REST maps them

onto HTTP methods applied to two URLs per resource, the **collection** and the **item**.

The endpoints of a resource

CRUD	Method + path	Request body	Success	Typical failures
Read (list)	GET /api/tasks	none	200 + array	400 bad query value
Read (one)	GET /api/tasks/:id	none	200 + the item	404
Create	POST /api/tasks	the new item (no id)	201 + Location + the item	400
Update (replace)	PUT /api/tasks/:id	the complete item	200 + the item	400 · 404
Update (partial)	PATCH /api/tasks/:id	only the changed fields	200 + the item	400 · 404
Delete	DELETE /api/tasks/:id	none	204, no body	404

```
// routes/books.js, mounted in app.js with app.use('/api/books', router)
router.get('/', listBooks);           // GET   /api/books      200 [ ... ]
router.post('/', createBook);         // POST  /api/books      201 + Location
router.get('/:id', getBook);          // GET   /api/books/:id  200 | 404
router.put('/:id', replaceBook);      // PUT   /api/books/:id  200 | 400 | 404
router.patch('/:id', patchBook);     // PATCH /api/books/:id  200 | 400 | 404
router.delete('/:id', deleteBook);   // DELETE /api/books/:id 204 | 404
```

Example. One lifecycle in the request builder below: GET /api/tasks lists five tasks (200); POST /api/tasks with {"title":"Study REST"} answers 201 and Location: /api/tasks/6; GET /api/tasks/6 reads it (200); PATCH /api/tasks/6 with {"done":true} changes one field (200); DELETE /api/tasks/6 removes it (204); a final GET /api/tasks/6 answers 404. Six requests, one URL pattern, and the status codes tell the whole story.

Common mistake. Sending updates to the collection, e.g. PUT /api/tasks with {"id":6, "done":true} in the body. The URL must name the resource being changed, so the id belongs in the path: PATCH /api/tasks/6. On a well-designed API the collection version answers 405.

Safe and idempotent: why retries matter

A **safe** method changes nothing on the server (GET); an **idempotent** method leaves the server in the same state whether it is sent once or many times (GET, PUT, DELETE). POST is neither, and PATCH is not guaranteed to be.

The five methods and retries

Method	Safe	Idempotent	Retrying after a timeout...
GET	yes	yes	is harmless
PUT	no	yes	sets the same full version again
DELETE	no	yes	leaves it deleted (the retry may answer 404)
PATCH	no	not guaranteed	is fine for "set field to X", not for "add 1"
POST	no	no	may create a duplicate, unless an idempotency key is used

Example. The train app sends POST /api/tasks with {"title":"Buy tickets"}, gets no answer and retries: the list now has two "Buy tickets". With the header Idempotency-Key: 9b2e... on both attempts, the server sees the key the second time and answers the same 201 Created with Location: /api/tasks/6, without creating task 7.

Common mistake. "DELETE is not idempotent, because the second call gets a 404 instead of a 204." Idempotence promises the same **effect on the server**, not the same status code. The response may differ; the state (the task is gone) does not.

PUT replaces, PATCH changes

PUT sends the **complete** new version of a resource and replaces it (fields you leave out go back to their defaults, or the request is rejected); **PATCH** sends **only the fields to change** and keeps all the others.

```
const task = { id: 2, title: 'Add full CRUD', done: false, userId: 1 };
const body = { done: true };           // what the client sent

// PUT: build the whole new version from the body (+ defaults)
const afterPut = {
  id: task.id,
```

```

    title: body.title,                                // undefined → a PUT without title must be a 400
    done: body.done ?? false,
    userId: body.userId ?? null,
  };
  console.log('PUT ', afterPut);

// PATCH: keep everything, overwrite only the allowed fields that were sent
const afterPatch = { ...task };
for (const key of ['title', 'done', 'userId']) {
  if (key in body) afterPatch[key] = body[key];
}
console.log('PATCH', afterPatch);

```

Example. Task 2 is { "title": "Add full CRUD", "done": false, "userId": 1 }. Leo renames it with PATCH /api/tasks/2 and {"title":"Add CRUD + tests"}. One second later Ana, whose screen still shows the old title, ticks it as done with PUT /api/tasks/2 and {"title":"Add full CRUD","done":true,"userId":1}. Leo's new title is gone. Had Ana sent PATCH with {"done":true}, both changes would be kept.

Common mistake. Writing PUT as a merge, e.g. `Object.assign(task, req.body)`. That is PATCH behaviour (fields not sent are kept), so the API no longer has a real PUT, and it also lets the body overwrite id or add fields you never meant to store. A PUT builds the new object from the allowed fields only.

Status codes and errors

Success codes: 200, 201 and 204

A successful call answers **200 OK** with a body, **201 Created** with a `Location` header after something was created, or **204 No Content** when there is nothing to send back.

Which success code

Situation	Code	Body	Header to add
GET a list or one item	200 OK	the data	(X-Total-Count for lists, if you use it)
POST created something	201 Created	the new resource	Location: /api/tasks/6
PUT / PATCH updated	200 OK	the resource as it is now	
DELETE removed	204 No Content	none	
List or search with no matches	200 OK	[]	

```

res.json(books);                                // 200 is the default status
res.status(201).location(`/api/books/${book.id}`).json(book); // created
res.json(updatedBook);                          // 200 after PUT / PATCH
res.status(204).end();                          // deleted, no body

```

Example. Reading `curl -i -X POST http://localhost:3000/api/tasks -H "Content-Type: application/json" -d '{"title":"Study"}'`: the first line HTTP/1.1 201 Created and the header Location: /api/tasks/6 already tell the client everything; the JSON body adds the id and createdAt the server chose.

Common mistake. Writing `res.status(204).json({ message: "Deleted" })` and expecting the client to show the message. A 204 never has a body: Express silently drops it. Either answer 204 with nothing, or 200 with a body.

Client errors: choosing the right 4xx

A **4xx** code tells the caller that **its request** must change, and each code names a different problem: unreadable or invalid (400), no such resource (404), wrong method (405), a clash with stored data (409), a body in the wrong format (415), or valid-looking data that breaks a rule (422, where an API uses it).

The 4xx codes of a REST API

Code	Meaning	Typical case	What the client must fix
400 Bad Request	Unreadable or invalid request	Malformed JSON; title missing (if the API uses 400 for validation); ?limit=abc	The syntax or the values

Code	Meaning	Typical case	What the client must fix
404 Not Found	Nothing at this URL	GET /api/tasks/999; /api/taks	The id or the path
405 Method Not Allowed	Path exists, method does not	DELETE /api/tasks (the whole collection)	Use a method from Allow
406 Not Acceptable	Cannot answer in the format asked for	Accept: text/html on a JSON-only API	The Accept header
409 Conflict	Clashes with the current state	Signing up with an email already registered	Different data, or change the state first
415 Unsupported Media Type	Body format not accepted	JSON body sent without Content-Type: application/json	The Content-Type header
422 Unprocessable Content	Readable but breaks a rule (if the API uses it)	{ "title": "" } on an API that keeps 400 for parse errors	The values

Example. Five versions of a sign-up, POST /api/users: with the body in plain text, **415**; with { "name": "Iris", } (trailing comma), **400** (cannot parse); with { "name": "", "email": "nope" }, **400** with two field errors (or 422 elsewhere); with a valid body but the email ana@example.com, which already exists, **409**; sent as PATCH /api/users, **405** with Allow: GET, POST.

Common mistake. Answering **500** for bad input. If invalid data reaches code that assumes it is valid, req.body.title.trim() throws "Cannot read properties of undefined" and the error handler sends 500, telling the client the **server** failed. Validate first and answer 400: the caller made the mistake and can fix it.

401, 403 or 404: who are you, and may you?

401 Unauthorized means the server does not know who you are (no token, a malformed one or an expired one); **403 Forbidden** means it knows and you are not allowed; some APIs answer **404** instead of **403** so as not to reveal that a resource exists.

Example. In the request builder, POST /api/users is for admins. With no Authorization header: **401** with WWW-Authenticate: Bearer. With Authorization: Bearer nonsense: **401** again (an unknown token is no identity). With Authorization: Bearer student-token: **403** ("you are a student"). With Authorization: Bearer admin-token and a valid body: **201**.

Common mistake. Sending 401 for "logged in but not allowed". The front end reacts to 401 by sending the user to the login page; they log in again, get 401 again, and loop forever. "Not allowed" is 403.

One error shape, and when to use 500

Every error answer should have the **right status code** and the **same JSON shape**, for example { "error": "message" } **plus an errors list** of { field, message } for validation, so a client handles every error with one piece of code; **500** is only for failures of the server itself.

```
// app.js: the LAST middleware. Every error leaves in the same shape.
app.use((err, req, res, next) => {
  const status = err.status || 500;
  if (status >= 500) console.error(err);           // details stay in the server log
  res.status(status).json({
    error: status >= 500 ? 'Internal server error' : err.message,
    ...(err.errors ? { errors: err.errors } : {}), // field errors, when there are any
  });
});
```

Example. Because the shape never changes, the client needs one function: const res = await fetch(url, options); if (!res.ok) { const { error, errors = [] } = await res.json(); errors.forEach((e) => showFieldError(e.field, e.message)); showBanner(error); }. It works for a 400 with field errors, a 404 and a 500 alike.

Common mistake. Answering `res.status(200).json({ success: false, error: "Not found" })`. Every tool that reads status codes now records a success, and `res.ok` is true in the browser. Put the meaning in the status code (404) and keep the body for the details.

The API contract

JSON in, JSON out: Content-Type and Accept

A JSON API reads a request body only when its Content-Type: application/json header says it is JSON, always answers with Content-Type: application/json, and can read the client's Accept header to see which formats it understands.

```
const book = { title: 'Dune', read: false, due: new Date(Date.UTC(2026, 9, 20)), notes: undefined };

console.log(JSON.stringify(book));           // the Date becomes an ISO string; undefined disappears
console.log(String({ title: 'Dune' }));      // what fetch sends if you forget JSON.stringify

const back = JSON.parse('{"title":"Dune","due":"2026-10-20T00:00:00.000Z"}');
console.log(typeof back.due);                // a string: JSON has no date type

try {
  JSON.parse("'title':'Dune'");              // single quotes are not JSON
} catch (e) {
  console.log('400 material:', e.name);
}
```

Example. The right fetch call: `fetch("/api/books", { method: "POST", headers: { "Content-Type": "application/json", Accept: "application/json" }, body: JSON.stringify(book) })`. Leave out the header and an Express 4 API sees `req.body` as `{}`; leave out `JSON.stringify` and the body is the text `[object Object]`, which no JSON parser can read.

Common mistake. Debugging the validation code when the real problem is a missing Content-Type header. If the API says a field is missing that you can see in your request, check the header first: `curl -i` (or the request builder) shows exactly what was sent.

The API contract vs the database row (DTOs)

A DTO (Data Transfer Object) is the shape of the data your API sends and accepts. It is deliberately different from the database row: it leaves out secrets and internal columns, and renames fields to the JSON convention.

```
// One row as the database returns it
const row = { id: 2, email: 'leo@example.com', password_hash: '$2b$10$Qm1x...',
  role: 'student', created_at: '2026-09-02T08:00:00.000Z', is_deleted: false };

// X Block-list: remove what you remembered to remove
const { password_hash, ...leaky } = row;
console.log(leaky);           // is_deleted and created_at leak, and so will any new column

// ✓ Allow-list: copy only the contract
const toUserDto = (r) => ({ id: r.id, email: r.email, role: r.role, createdAt: r.created_at });
console.log(toUserDto(row));
```

Example. In the request builder, the server state panel shows the stored user rows with `password_hash`, `created_at` and `is_deleted`. Send `GET /api/users`: every user comes back as `{ "id", "name", "email", "role", "createdAt" }`. Create a user with a password: the 201 body does not contain the password or its hash.

Common mistake. Sending the row straight out, `res.json(user)`, or fixing it with `delete user.password_hash`. The first leaks the hash; the second is a block-list that forgets the next secret column (and, with a shared object, deletes the hash from your data too).

Documenting the API: OpenAPI

An **OpenAPI** document (formerly called **Swagger**) describes every endpoint of an API in one **YAML** or **JSON** file: **paths**, **methods**, **parameters**, **request** and **response** bodies and **status codes**, in a form that people and programs can both read.

YAML

```
openapi: 3.0.3
info: { title: Books API, version: 1.0.0 }
paths:
  /api/books:
    post:
      summary: Add a book
      requestBody:
        required: true
        content:
          application/json:
            schema: { $ref: '#/components/schemas/NewBook' }
      responses:
        '201':
          description: Created. Location holds the URL of the new book.
          content:
            application/json:
              schema: { $ref: '#/components/schemas/Book' }
        '400': { description: Validation failed (error + errors list) }
  /api/books/{id}:
    get:
      parameters:
        - { in: path, name: id, required: true, schema: { type: integer } }
      responses:
        '200': { description: OK }
        '404': { description: No book with that id }
components:
  schemas:
    NewBook:
      type: object
      required: [title]
      properties:
        title: { type: string, minLength: 1, maxLength: 120 }
        read: { type: boolean, default: false }
    Book:
      allOf:
        - $ref: '#/components/schemas/NewBook'
        - properties:
            id: { type: integer }
            createdAt: { type: string, format: date-time }
```

Example. A front-end developer asks: "what do I get if the title is missing?" Without a spec they try it, or read your code. With the spec they open `/api/books` → `post` → `responses` and see `400`, "Validation failed (error + errors list)", plus the `NewBook` schema telling them that `title` is required and at most 120 characters long.

Common mistake. Writing the documentation at the end and never touching it again. Documentation that disagrees with the API is worse than none, because people trust it. Update the spec, or the `README` table, in the same commit as each change to a route.

Building and testing

CRUD with an in-memory array

Before a database, an API can keep its data in a JavaScript **array** in memory: each handler turns the `id` from the path into a number, finds the item, answers **404** if it is missing, and changes the array in place, behind function names that a database layer can keep later.

```
let books = [{ id: 1, title: 'Dune', read: true }];
let nextId = 2;
```

```
function getBook(req, res) {
  const id = Number(req.params.id);           // "1" → 1
  const book = books.find((b) => b.id === id);
  if (!book) return res.status(404).json({ error: `Book ${id} not found` });
  res.json(book);
}

function deleteBook(req, res) {
  const i = books.findIndex((b) => b.id === Number(req.params.id));
  if (i === -1) return res.status(404).json({ error: 'Book not found' });
  books.splice(i, 1);
  res.status(204).end();
}
```

```
const books = [{ id: 1, title: 'Dune' }];
console.log(books.find((b) => b.id === '1'));           // undefined: "1" is not 1
console.log(books.find((b) => b.id === Number('1')));    // found

let nextId = 2;
const body = { id: 1, title: 'Hacked', isAdmin: true }; // what a client could send
console.log({ id: nextId++, ...body });                 // X the body overwrote the id
console.log({ id: nextId++, title: body.title });        // ✓ only the allowed fields
```

Example. DELETE /api/books/7 when book 7 does not exist: `findIndex` returns -1, the handler answers 404 and returns. Without that check, `books.splice(-1, 1)` would silently remove the **last** book in the array and answer 204: a wrong item deleted and a success reported.

Common mistake. Forgetting `return` before an early answer: `if (!book) res.status(404).json(...)` followed by `res.json(book)`. The second call crashes with `ERR_HTTP_HEADERS_SENT`. Every early answer starts with `return`.

Validating input: 400 with field errors

Validation checks every field of the request body (present, right type, right length or range, allowed value) before anything is stored, and rejects a bad request with 400 and a list of **all the problems, each tied to its field.**

```
function validateBook(body, { partial = false } = {}) {
  const errors = [];
  const has = (k) => body[k] !== undefined;
  if (!partial || has('title')) {
    if (typeof body.title !== 'string' || !body.title.trim()) {
      errors.push({ field: 'title', message: 'title is required' });
    }
  }
  if (has('rating') && !(Number.isInteger(body.rating) && body.rating >= 1 && body.rating <= 5)) {
    errors.push({ field: 'rating', message: 'rating must be a whole number from 1 to 5' });
  }
  return { valid: errors.length === 0, errors };
}

console.log(validateBook({ title: 'Dune', rating: 5 }));
console.log(validateBook({ title: ' ', rating: '5' }));           // two errors, both reported
console.log(validateBook({ rating: 4 }, { partial: true }));       // PATCH: title not needed
```

Example. POST /api/books with `{ "title": " ", "rating": "5" }` answers 400 with `{ "error": "Validation failed", "errors": [ { "field": "title", "message": "title is required" }, { "field": "rating", "message": "rating must be a whole number from 1 to 5" } ] }`, and the collection has the same length as before. The client shows each message under its input.

Common mistake. Checking only `if (!req.body.title)`. It accepts `"title": 123` and `"title": " "`, says nothing about the other fields, and reports one problem at a time, so the user fixes, resubmits and gets the next error, again and again.

Testing by hand: curl, Postman, Thunder Client

curl sends any HTTP request from the terminal and prints the response; four flags, **-i** (show status and headers), **-X** (method), **-H** (header) and **-d** (body), cover every CRUD call. GUI clients such as Postman or Thunder Client do the same with forms and saved collections.

shell (Git Bash, macOS, Linux)

```
curl -i http://localhost:3000/api/tasks
curl -i "http://localhost:3000/api/tasks?done=false&sort=createdAt&order=desc"
curl -i -X POST http://localhost:3000/api/tasks \
  -H "Content-Type: application/json" \
  -d '{"title":"Study REST"}'
curl -i -X PATCH http://localhost:3000/api/tasks/2 \
  -H "Content-Type: application/json" -d '{"done":true}'
curl -i -X DELETE http://localhost:3000/api/tasks/2
curl -i -H "Authorization: Bearer $TOKEN" http://localhost:3000/api/me
```

Example. The POST above prints HTTP/1.1 201 Created, then headers including Location: /api/tasks/6 and Content-Type: application/json; charset=utf-8, an empty line, and the JSON body. That is the whole contract of a create in four lines; without **-i** you would only see the JSON and could not tell 201 from 200. Note the quotes around the URL with &: unquoted, the shell treats & as "run in the background".

Common mistake. Typing `curl -X POST ... -H "Content-Type: application/json"` in Windows PowerShell 5.1 and getting "Cannot bind parameter Headers": that is Invoke-WebRequest speaking, not curl. Use `curl.exe`, or Git Bash.

Automated API tests with supertest

An automated API test sends a request to your Express app **inside the test process with supertest**, then checks the status code, the body and, for writes, the change in state; a test runner (**Jest**, or Node's built-in `node:test`) runs them all with `npm test`.

```
// tests/books.test.js (Jest + supertest)
const request = require('supertest');
const app = require('../src/app');          // the app, NOT server.js

test('POST /api/books → 201 with Location and a server id', async () => {
  const res = await request(app).post('/api/books').send({ title: 'Dune' });
  expect(res.status).toBe(201);
  expect(res.body).toMatchObject({ title: 'Dune' });
  expect(res.headers.location).toBe(`/api/books/${res.body.id}`);
});

test('POST without a title → 400, and nothing is added', async () => {
  const before = await request(app).get('/api/books');
  const res = await request(app).post('/api/books').send({});
  expect(res.status).toBe(400);
  const after = await request(app).get('/api/books');
  expect(after.body.length).toBe(before.body.length);
});
```

Example. Run `npm test` and Jest prints one line per test, ✓ or ✗. A failure shows Expected: 201, Received: 200: you forgot `res.status(201)` in the create handler. You fix one line, run again, and every other route is checked again for free.

Common mistake. Calling `app.listen(3000)` inside `app.js`. Every test file that imports the app now opens port 3000 (EADDRINUSE when two files run) and Jest never finishes because a server is still listening. Export the app from `app.js` and listen only in `server.js`.